Object Deep dive

```
class Student {
    String name;   → 4
    int age;       → 4
    int rollNo;    → 4
    String college; → 4
}
```

= 16 bytes ✓

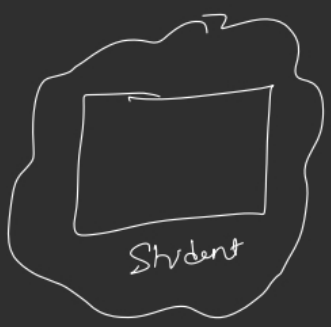
```
Student s1 = new Student();
```

4 bytes (for)
8 bytes

12 bytes
+
16 bytes
+
4 bytes
= 32 bytes ✓

Object Size

1. Header Size → Object Headers
2. Exact fields
3. Padding



Object Headers

↓

Metadata

- Mark words
  - lock
  - Synchronization
  - Garbage Collection
- Class Pointer → 4 bytes ✓ or 8 bytes

ok

8 bytes

= 12 bytes

Exact data

→ 16 bytes

```
class Student {
    String name;   → 4
    int age;       → 4
    int rollNo;    → 4
    String college;  → 4
}
```

Padding

↳ 8 bytes Multiple

CPU →

= Object Header + Exact data

= 12 + 16

= 28 bytes ✓

4 bytes → padding

32 bytes

28 bytes

4 bytes

```
Class Person {
    byte age;
}
```

= 18 bytes

= Object Metadata

12 bytes

= Exact data

= 1 byte

13 bytes

3 bytes Padding

Call by Value

```
Class Demo {
    main() {
        int x = 4;
        int y = 5;
        System (x, y);
        addTen(x, y);
        System (x, y);
    }
    addTen (int x, int y){
        x = x + 10;
        y = y + 10;
    }
}
```

4 (x), 5 (y)

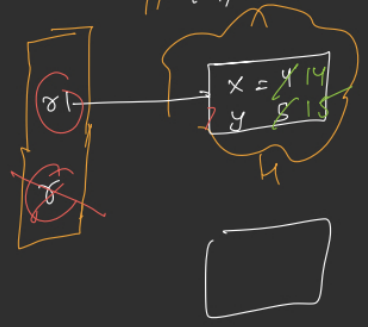
// 4, 5

// 4, 5

Call by Reference

```
class Demo {
    main() {
        Random r1 = new Random(4,5);
        System.out.(r1.x, r1.y);
        addTen(r1);
        System.out.(r1.x, r1.y);
    }
    addTen(Random r) {
        r.x = r.x + 10;
        r.y = r.y + 10;
    }
}
class Random {
    int x;
    int y;
}
```

// 4, 5
// 14, 15



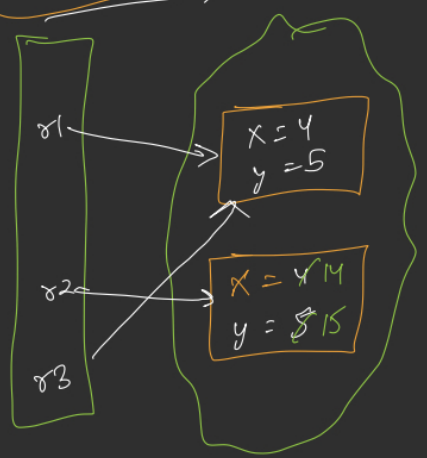
Java ?

- Call by Value ?
- Call by Reference ?

Call by Value ✓

```
Random r1 = new Random(4,5);
Random r2 = new Random(r1);
Random r3 = r1;
```

Deep Copy
Shallow Copy

r3.x = 14

```
class Random {
    int x;
    int y;
    Random(int x, int y) {
        this.x = x;
        this.y = y;
    }
    Random(Random r) {
        this.x = r.x;
        this.y = r.y;
    }
}
```